

Raport z listy 10

Modele regresji i ich zastosowania

Tadeusz Jagnieski, album 277522

2026-05-13

Spis treści

1	Generowanie zbioru danych do regresji logistycznej	1
1.1	Część 1.	1
1.2	Część 2.	2
2	Konstrukcja modelu regresji logistycznej	4
2.1	Zadanie 1: Wczytać do pakietu statystycznego dane z utworzonego pliku. . .	4
2.2	Zadanie 2	7
2.3	zadanie 3	7
2.4	Zadanie 4	9

1 Generowanie zbioru danych do regresji logistycznej

Model regresji logistycznej jest postaci:

$$\log \frac{p(\mathbf{x})}{1 - p(\mathbf{x})} = x\beta^T$$

gdzie $p(\mathbf{x}) = \mathbb{P}(Y = 1 \mid \mathbf{X} = \mathbf{x})$ to prawdopodobieństwo sukcesu, $\mathbf{x}$ to wektor cech, a β to wektor nieznanych parametrów w modelu.

Reguła klasyfikacyjna jest postaci:

$$d^*(\mathbf{x}) = \begin{cases} 1, & \text{gdy } p(\mathbf{x}) > \pi_0, \\ 0, & \text{gdy } p(\mathbf{x}) \leq \pi_0, \end{cases}$$

gdzie π_0 jest pewną stałą z przedziału $(0, 1)$.

1.1 Część 1.

Generujemy $n = 300$ obserwacji a liczba zmiennych objaśniających jest równa 20.

```

# ustawiamy ziarno
set.seed(67)

# rozmiar próby
n <- 300

# liczba zmiennych objaśniających
p<-20

# wektor współczynników beta
beta <- numeric(20)
beta[1:10] <- runif(10, -3, 3)

```

Współczynniki β w modelu to

```
print(cbind(i=1:20, beta_i=beta))
```

```

##      i      beta_i
## [1,]  1  2.3319250
## [2,]  2 -2.0234970
## [3,]  3 -0.2970055
## [4,]  4  2.5283343
## [5,]  5 -1.9850535
## [6,]  6  0.8150098
## [7,]  7 -0.1893066
## [8,]  8  0.4717027
## [9,]  9 -2.2413964
## [10,] 10  1.2876638
## [11,] 11  0.0000000
## [12,] 12  0.0000000
## [13,] 13  0.0000000
## [14,] 14  0.0000000
## [15,] 15  0.0000000
## [16,] 16  0.0000000
## [17,] 17  0.0000000
## [18,] 18  0.0000000
## [19,] 19  0.0000000
## [20,] 20  0.0000000

```

1.2 Część 2.

1.2.1 Podpunkt (a)

Dla $i = 1, 2, \dots, 20$ przyjmujemy, że $x_{i1} = 1$ a następnie generujemy, za pomocą standardowego rozkładu normalnego $\mathcal{N}(0, 1)$, wartości $x_{i2}, x_{i3}, \dots, x_{i20}$ dla pozostałych $p - 1 = 19$ zmiennych objaśniających $X_2, X_3, \dots, X_{20}$

```
# ustawianie ziarna
set.seed(123)

# generowanie obserwacji dla X1 (same jedynki)
X1 <- rep(1,n)

# generowanie obserwacji dla X2,...,X20 z rozkładu normalnego N(0,1)
X2...20 <- matrix(rnorm(n*(p-1)), nrow=n, ncol=p-1)

# łączymy kolumnowo [X1] oraz [X2...20] w macierz X
X <- cbind(X1, X2...20)
```

1.2.2 Podpunkt (b)

Generujemy wartości $y_i, i = 1, 2, \dots, 300$ binarnej zmiennej objaśniającej Y , która przyjmuje wartości 0 oraz 1 z prawdopodobieństwami, odpowiednio, $\pi(\mathbf{x}_i)$ oraz $1 - \pi(\mathbf{x}_i)$, gdzie $\mathbf{x}_i = (x_{i1}, \dots, x_{ip})$ oraz

$$\pi(\mathbf{x}_i) = \mathbb{P}(Y = 1 \mid X_1 = x_{i1}, \dots, X_p = x_{ip}) = \frac{1}{1 + \exp(-(\beta_1 x_{i1} + \dots + \beta_p x_{ip}))}$$

```
# ustawiamy ziarno
set.seed(67)

# zapisujemy iloczyn beta_1 * x_{i1} + ... + beta_p * x_{ip} jako [lin_pred]
lin_pred <- X%*%beta

# zapisujemy p-stwo 1/(1+exp(...)) z wykorzystaniem [lin_pred] jako [prob]
prob <- 1/(1 + exp(-lin_pred))

# generujemy [y] jako n obserwacji z rozkładu dwumianowego B(1,prob)
Y<-rbinom(n,1,prob)

# łączymy [y] oraz [X] w ramkę danych [df]
df<-as.data.frame(cbind(X,Y))

# zmieniamy nazwy kolumn tej ramki danych [df] na X1,...,X20,Y
colnames(df)<-c(paste0("x",1:p),"Y")

# i zapisujemy [df] do pliku "dane_logistyczne.csv" z row.names=FALSE
write.csv(df, file="dane_logistyczne.csv", row.names=FALSE)

# i wczytujemy ten plik jako [dane]
dane<-read.csv(file="dane_logistyczne.csv",sep="," ,dec=".",check.names=F)
```

2 Konstrukcja modelu regresji logistycznej

2.1 Zadanie 1: Wczytać do pakietu statystycznego dane z utworzonego pliku.

2.1.1 podpunkt (a):

Cały zbiór danych zawiera 300 obserwacji, które podzielimy w proporcji 70 : 30 na część uczącą i część testową za pomocą funkcji `createDataPartition()` z pakietu `caret`.

```
# ustawiamy ziarno
set.seed(123)

# załączamy pakiet caret
library(caret)

# wybieramy indeksy obserwacji do zbioru treningowego
trainIndex <- createDataPartition(dane$Y, p=0.7, list=FALSE)

# tworzymy część treningową
train <- dane[trainIndex,]

# tworzymy część testową
test <- dane[-trainIndex,]
```

2.1.2 podpunkt (b):

Modele regresji logistycznej stworzymy przy pomocy funkcji `glm()`.

```
# model regresji logistycznej ze wszystkimi zmiennymi
model.full <- glm(Y~., family = "binomial", data=train[, -1])

# model regresji logistycznej tylko ze stałą
model.null <- glm(Y~1, family = "binomial", data=train[, -1])
```

2.1.3 podpunkt (c):

Do przeprowadzenia wspomnianego w zadaniu testu ilorazu wiarygodności wykorzystamy funkcję `anova()`.

```
lrt.test <- anova(model.null, model.full, test = "LRT")
p.value <- lrt.test$`Pr(>Chi)`[2]
```

Na podstawie testu opartego na ilorazie wiarygodności na poziomie istotności $\alpha = 0.05$ odrzucamy hipotezę H_0 mówiącą, że $\beta_2 = \beta_3 = \dots = \beta_{20} = 0$, więc przynajmniej jedna jest niezerowa, więc przynajmniej jedna beta ma liniowy wpływ na model regresji P-wartość tego testu to $7.8805368 \times 10^{-26}$.

2.1.4 podpunkt (d):

Estymatory $\hat{\beta}_i$ oraz ich błędy standardowe SE_{β_i} otrzymujemy za pomocą podsumowania `summary()` modelu pełnego i są podane w tabeli poniżej.

```
# oszacowania współczynników, błędy standardowe tych oszacowań i p-wartości  
# za pomocą summary(...)[["coefficients"]]
```

```
summary(model.full)[["coefficients"]]
```

##	Estimate	Std. Error	z value	Pr(> z)
## (Intercept)	2.63549382	0.4901321	5.3771087	7.569147e-08
## x2	-2.23894297	0.4853918	-4.6126514	3.975647e-06
## x3	-0.32746044	0.3043724	-1.0758547	2.819922e-01
## x4	2.75485279	0.4961361	5.5526155	2.814267e-08
## x5	-2.14142827	0.4583655	-4.6718797	2.984557e-06
## x6	0.69485826	0.3772057	1.8421202	6.545757e-02
## x7	0.21044113	0.3347957	0.6285658	5.296334e-01
## x8	0.33429974	0.2895477	1.1545586	2.482712e-01
## x9	-2.07677245	0.4661299	-4.4553509	8.375609e-06
## x10	1.23007321	0.4003810	3.0722563	2.124472e-03
## x11	-0.07602405	0.3282388	-0.2316120	8.168394e-01
## x12	-0.20613806	0.3670579	-0.5615955	5.743917e-01
## x13	0.07245934	0.3005601	0.2410810	8.094923e-01
## x14	0.41741681	0.3730222	1.1190133	2.631345e-01
## x15	-0.19836167	0.2567424	-0.7726098	4.397533e-01
## x16	0.30420964	0.3923721	0.7753091	4.381570e-01
## x17	-0.59799786	0.3394415	-1.7617109	7.811815e-02
## x18	0.43610450	0.3713963	1.1742297	2.403030e-01
## x19	0.46046515	0.3526628	1.3056812	1.916610e-01
## x20	0.05876807	0.2840368	0.2069030	8.360856e-01

Przedziały ufności otrzymujemy za pomocą funkcji `confint()` i są podane w kolejnej tabeli.

```
# ustawiamy ziarno
```

```
set.seed(123)
```

```
#tworzenie przedziałów ufności dla [model.full] za pomocą confint()  
przedzialy <- confint(model.full)
```

```
# wyświetlenie stworzonych przedziałów ufności  
przedzialy
```

##	2.5 %	97.5 %
## (Intercept)	1.79420752	3.74888607
## x2	-3.32475616	-1.39268046
## x3	-0.96031878	0.25305376

```
## x4          1.90607540  3.88623139
## x5         -3.17629177 -1.35317760
## x6         -0.01900988  1.48263651
## x7         -0.44168878  0.88974753
## x8         -0.22400757  0.92930934
## x9         -3.12207921 -1.26504559
## x10         0.52865871  2.11986901
## x11        -0.71843716  0.58101360
## x12        -0.94255429  0.51333970
## x13        -0.52487817  0.66879061
## x14        -0.28547693  1.19892267
## x15        -0.72363070  0.29845771
## x16        -0.46225529  1.09582487
## x17        -1.30057763  0.05177103
## x18        -0.29208134  1.18708685
## x19        -0.20220021  1.20571942
## x20        -0.50777192  0.62233104
```

Do zweryfikowania hipotezy $H_0 : \beta_i = 0$ przeciwko hipotezie alternatywnej $H_1 : \beta_i \neq 0$ można wykorzystać tabelę z przedziałami ufności.

Otrzymujemy z niej, że zmienne x2, x4, x5, x9, x10 mają istotny wpływ na zmienną Y , ponieważ nie zawierają zera w przedziale ufności i p-wartość jest mniejsza niż 0.05.

Alternatywnie: można wykorzystać wyznaczone p-wartości tych testów:

```
which(summary(model.full)[["coefficients"]][,4]<0.05)
```

```
## (Intercept)          x2          x4          x5          x9          x10
##             1           2           4           5           9          10
```

2.1.5 podpunkt (e):

```
data.frame(beta = beta,
            beta.est = model.full$coefficients,
            istotny = ifelse(summary(model.full)[["coefficients"]][,4]<0.05,
                             "TAK",
                             "nieistotna"))
```

```
##          beta    beta.est    istotny
## (Intercept) 2.3319250 2.63549382      TAK
## x2         -2.0234970 -2.23894297      TAK
## x3         -0.2970055 -0.32746044 nieistotna
## x4          2.5283343  2.75485279      TAK
## x5         -1.9850535 -2.14142827      TAK
## x6          0.8150098  0.69485826 nieistotna
## x7         -0.1893066  0.21044113 nieistotna
```

```
## x8          0.4717027  0.33429974 nieistotna
## x9         -2.2413964 -2.07677245          TAK
## x10         1.2876638  1.23007321          TAK
## x11         0.0000000 -0.07602405 nieistotna
## x12         0.0000000 -0.20613806 nieistotna
## x13         0.0000000  0.07245934 nieistotna
## x14         0.0000000  0.41741681 nieistotna
## x15         0.0000000 -0.19836167 nieistotna
## x16         0.0000000  0.30420964 nieistotna
## x17         0.0000000 -0.59799786 nieistotna
## x18         0.0000000  0.43610450 nieistotna
## x19         0.0000000  0.46046515 nieistotna
## x20         0.0000000  0.05876807 nieistotna
```

Od 11 do 20 wszystkie model prawidłowo zidentyfikował jako nieistotne, ale niestety uznał też za nieistotne zmienne x3, x6, x7, x8 a teoretycznie powinny być istotne, występuje też jedna zmiana znaku w przypadku zmiennej x7, ale jest ona nieistotna.

2.2 Zadanie 2

Ilorazy szans to eksponenty z estymatorów współczynników tzn. $\exp(\hat{\beta}_i)$.

Przykładowo, dla zmiennych x4 oraz x5 te ilorazy szans to, odpowiednio, 15.71873 oraz 0.1174869

Oznacza to, że przy wzroście x4 o 1 szanse rosną o około 1472%, natomiast wzrost x5 o jeden powoduje spadek szans o około 88.25%

Jeśli obie te zmienne wzrosną o 1 to wtedy szanse rosną o około 84.67%, bo ich wspólny iloraz szans to 1.846745 ($\approx 15.71873 \cdot 0.1174869$)

2.3 zadanie 3

Konstruujemy macierz pomyłek postaci

		Actual	
		0	1
Predicted	0	TN	FN
	1	FP	TP

Wyznaczamy miary zdolności predykcyjnej modelu:

- $Sensitivity = \frac{TP}{TP+FN}$
- $Specificity = \frac{TN}{TN+FP}$

- $FPR = \frac{FP}{TN+FP}$
- $FNR = \frac{FN}{TP+FN}$
- $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$

```
# predykcje [model.full] na zbiorze [test] za pomocą funkcji predict()
# z argumentem type="response"
predict.test <- predict(model.full, newdata = test, type = "response")

#funkcja wyznaczająca macierz pomyłek dla danego poziomu odcięcia threshold

conf.matrix.metrics<-function(threshold){

  # przewidywana klasa, zgodnie z regułą klasyfikacyjną d*
  pred.class <- ifelse(predict.test > threshold, 1, 0)

  # macierz pomyłek za pomocą table()
  # dla Predicted=pred.class i Actual=test$Y
  cm <- table(Predicted=pred.class, Actual=test$Y)

  # wskaźniki
  sensitivity <- cm[2,2] / sum(cm[,2])
  specificity <- cm[1,1] / sum(cm[,1])
  fpr <- cm[2,1] / sum(cm[,1])
  fnr <- cm[1,2] / sum(cm[,2])
  acc <- (cm[1,1] + cm[2,2]) / sum(cm)

  # lista zwracanych wyników
  list(cm=cm,
       sensitivity = sensitivity,
       specificity = specificity,
       fpr = fpr,
       fnr = fnr,
       accuracy=acc)}

# macierz pomyłek [M1] dla threshold=0.5
M.1 <- conf.matrix.metrics(0.5)

# macierz pomyłek [M2] dla threshold=0.7
M.2 <- conf.matrix.metrics(0.7)
```

Macierze pomyłek:

```
print(M.1$cm)
```

```
##           Actual
```

```
## Predicted  0  1
##           0 21  7
##           1  7 55
```

```
print(M.2$cm)
```

```
##           Actual
## Predicted  0  1
##           0 22 13
##           1  6 49
```

Wskaźniki porównujące modele:

```
acc1<-M.1$accuracy
acc2 <- M.2$accuracy
cbind(M1 = M.1[2:6], M2 = M.2[2:6])
```

```
##           M1      M2
## sensitivity 0.8870968 0.7903226
## specificity 0.75      0.7857143
## fpr         0.25      0.2142857
## fnr         0.1129032 0.2096774
## accuracy    0.8444444 0.7888889
```

Lepszy jest niższy treshhold na poziomie 0.5 ponieważ jego accuracy wynosi 0.8444444, a drugiego 0.7888889 i ogólnie model lepiej sobie radzi w sensitivity i fpr, chociaż to akurat lepiej jakby było mniejsze. M2 jest nieco lepszy w specificity i znacząco więcej ma w fnr.

2.4 Zadanie 4

```
# załączamy pakiet <pROC>
library(pROC)

# tworzymy krzywą ROC [roc.object] za pomocą funkcji roc()
# dla [test$Y] oraz [predict.test]
roc.object <- roc(test$Y, predict.test)

# wyznaczmy AUC dla [roc.object] za pomocą funkcji auc()
auc(roc.object)
```

```
## Area under the curve: 0.9407
```

Wartość wychodzi wysoka więc jest to dobry model ponieważ chcemy aby było jak najbliżej jedynki no i jest zdecydowanie lepsze niż klasyfikator losowy, którego pole wynosi 0.5.

```
# rysujemy wykres krzywej ROC [roc.object] za pomocą
plot(roc.object, col="blue")
```

